

Tugas Kecerdasan Komputasional dan Pembelajaran Mesin

Implementasi *Multi Layer Perceptron* (MLP) pada dataset UCI



Oleh:

Fawwaz Rif'at Revista (25/565782/PPA/07130)

**PROGRAM MAGISTER ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
2026**

I. PENDAHULUAN

Gagal jantung adalah kondisi medis di mana jantung tidak lagi mampu memompa darah dengan optimal untuk memenuhi kebutuhan oksigen tubuh, baik saat beristirahat maupun beraktivitas [1]. Menurut ahli, hal ini disebabkan oleh kerusakan struktur jantung atau gangguan pada fungsinya sehingga tekanan di dalam jantung meningkat. Masalah ini dapat dikatakan cukup serius karena angka kematian pasien bahkan bisa mencapai 50% dalam waktu lima tahun setelah pertama kali didiagnosis [2]. Oleh karena itu, deteksi risiko dini sangatlah penting untuk menyelamatkan pasien.

Multilayer Perceptron (MLP) dapat digunakan untuk membantu dokter dalam memprediksi risiko ini dengan lebih akurat. MLP adalah salah satu jenis Jaringan Saraf Tiruan (JST) yang sangat pintar dalam melihat pola-pola rumit pada data pasien yang sulit dibaca oleh metode biasa [3]. Namun, agar MLP bisa bekerja dengan baik, data pasien masuk ke dalam tahap pra-pemrosesan terlebih dahulu agar datanya menjadi “bersih”, sehingga memungkinkan hasilnya juga akan baik. Selain pra-pemrosesan data, pada MLP terdapat beberapa parameter yang dapat digunakan untuk meningkatkan performa model jika dipilih dengan tepat, parameter tersebut seperti *solver*, *activation*, *batch size*, dll

Berdasarkan alasan tersebut, eksperimen ini dilakukan dengan beberapa skenario untuk menguji dan mendapatkan kombinasi terbaik model dalam memprediksi gagal jantung. Eksperimen ini juga dapat dikatakan sebagai studi komparatif karena melakukan uji perbandingan terhadap beberapa parameter untuk mendapatkan kombinasi parameter terbaik.

II. DATASET, SKENARIO, DAN PARAMETER

Link Github :

A. Dataset

Dataset didapatkan dari *website* UCI Machine Learning Repository dengan link sebagai berikut : <https://archive.ics.uci.edu/dataset/519/heart+failure+clinical+records>. Dataset ini merupakan data tabular yang terdiri dari 12 fitur dan 299 baris. Berikut penjelasan mengenai fitur yang ada dalam dataset ini :

- 1 age : umur pasien dalam tahun
- 2 anaemia : statu anemia (boolean)
- 3 creatinine phosphokinase (CPK) : Kadar enzim CPK di dalam darah (mcg/L)
- 4 diabetes: status diabetes (boolean)
- 5 ejection fraction : persentase darah yang keluar dari jantung pada setiap kontraksi (persentase)
- 6 high blood pressure : status hipertensi (boolean)
- 7 platelets : jumlah trombosit dalam darah (kiloplatelets/mL)
- 8 sex : jenis kelamin (biner)
- 9 serum creatinine : kadar serum kreatinin dalam darah (mg/dL)
- 10 serum sodium: kadar serum sodium dalam darah (mEq/L)
- 11 smoking : status merokok (boolean)
- 12 time : jumlah hari setelah didiagnosis hingga kejadian terakhir dicatat (hari)
- 13 [target] death event: if the patient died during the follow-up period (boolean)

B. Skenario

Berikut beberapa skenario yang akan dilakukan dalam eksperimen ini, yaitu :

1. Menggunakan 3 *solver* untuk menguji masing-masing solver yang terdiri dari *adam*, *lbfgs*, dan *sgd*.
2. Menguji 4 jumlah iterasi yang berbeda, yaitu 200 (default), 500, 750, dan 1000.
3. Menguji akurasi pada dataset dengan dan tanpa *scaling*. Skenario ini untuk membuktikan pernyataan “Tanpa proses *scaling*, variabel dengan angka yang besar (seperti tekanan darah) akan "menenggelamkan" variabel dengan angka kecil lainnya, sehingga hasil prediksi menjadi tidak akurat” [4].

C. Parameter

1. *solver*
Solver yang diuji ada 3, yaitu *adam*, *lbfgs*, dan *sgd*.
2. *hidden_layer_sizes*
Jumlah *hidden layer* yang digunakan adalah 2 dengan jumlah neuron 20 dan 10.
3. *max_iter*
Jumlah iterasi yang dilakukan ada 4, yaitu 200 (default), 500, 750, dan 1000.
4. *batch_size*
Batch size yang digunakan tidak menggunakan nilai defaultnya (200), tetapi menggunakan nilai 32.
5. *Output layer activation function*
Activation function yang digunakan pada *output layer* adalah sigmoid karena label targetnya adalah biner, tetapi parameter tersebut tidak ditulis didalam kode karena scikit learn sudah otomatis mengaturnya menjadi sigmoid.

III. HASIL DAN ANALISIS

1. Tanpa *scaling*

```
Model TANPA scaling
Akurasi MLP Train      : 70.29%
Akurasi MLP Tanpa Scaling Test : 58.33%
Precision Train        : 0.00%
Precision Test         : 0.00%
Recall Train           : 0.00%
Recall Test            : 0.00%
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:546: ConvergenceWarning: lbfgs failed to converge (status=2):
ABNORMAL: .

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
self.n_iter_ = _check_optimize_result("lbfgs", opt_res, self.max_iter)
```

Model tanpa *scaling* menghasilkan akurasi yang rendah, yaitu 58,3%, hal ini selaras dengan pernyataan “Tanpa proses *scaling*, variabel dengan angka yang besar (seperti tekanan darah) akan "menenggelamkan" variabel dengan angka kecil lainnya, sehingga hasil prediksi menjadi tidak akurat” [4].

Selain hasilnya *overfit*, juga terdapat *warning* yang memberikan peringatan terdapat 2 masalah, yaitu jumlah iterasi yang terlalu kecil dan perbedaan *range* atau skala nilai numerik antar yang fitur sangat jauh. Perbedaan *range* nilai yang sangat jauh ini menyebabkan beberapa fitur dianggap lebih penting dibandingkan fitur lainnya karena memiliki nilai yang lebih besar secara angka, padahal maknanya angka yang lebih besar tidak selalu berarti lebih penting. Sebagai contoh, fitur

platelets memiliki nilai hingga ratusan ribu, sedangkan *serum creatinine* hanya memiliki nilai satuan padahal kedua fitur itu sama pentingnya.

Skor *accuracy*, *precision*, dan *recall* yang didapatkan dapat dikatakan “jelek” karena *accuracy overfit*, sedangkan *precision* dan *recall* 0%. Skol 70% pada *accuracy train* kemungkinan karena 70% label adalah 0 dan model juga mengalami *warning* dan disarankan untuk melakukan *scaling* atau menambah jumlah iterasi. Setelah jumlah iterasi ditambah, pada awalnya 200 (default) hingga 100000, *warning* tersebut masih tetap muncul, sehingga harus dicoba untuk dilakukan *scaling*.

2. Dengan *scaling*

Scaling dilakukan dengan metode *standard scaler* karena metode ini tetap menjaga persebaran datanya ketika terdapat *outlier* ekstrim. Berbeda dengan *min max scaler* yang memaksa nilai berada pada rentang 0 – 1, sehingga menghilangkan nilai atau maknanya. Berikut hasil pada dataset yang telah dilakukan *scaling* :

```
Model DENGAN scaling
HASIL EVALUASI DENGAN SCALING
Akurasi Train : 100.00%
Akurasi Test : 66.67%
Precision Train : 100.00%
Precision Test : 61.90%
Recall Train : 100.00%
Recall Test : 52.00%
```

Berdasarkan hasil diatas, dapat dilihat bahwa sudah tidak terlalu jelek dibandingkan model sebelumnya yang tidak dilakukan *scaling*. Selain itu, tidak muncul lagi *warning*, sehingga artinya yang menjadi masalah sebelumnya adalah datasetnya tidak dilakukan *scaling* sebelum dilatih.

3. Berbagai skenario

Pada eksperimen ini dilakukan juga berbagai skenario untuk melihat hasil dan menganalisis perbedaan pada tiap skenario. *Activation function* yang digunakan adalah ReLU karena

Berikut hasil yang didapatkan :

Batch_size = 20

Solver	Iter	Tr_Acc(Raw)	Ts_Acc(Raw)	Tr_Prc(Raw)	Ts_Prc(Raw)	Tr_Rec(Raw)	Ts_Rec(Raw)	Tr_Acc(Scale)	Ts_Acc(Scale)	Tr_Prc(Scale)	Ts_Prc(Scale)	Tr_Rec(Scale)	Ts_Rec(Scale)
ADAM	200	69.0%	73.3%	51.3%	56.0%	79.2%	73.7%	100.0%	66.7%	100.0%	46.7%	100.0%	36.8%
ADAM	500	69.0%	73.3%	51.3%	56.0%	79.2%	73.7%	100.0%	71.7%	100.0%	57.1%	100.0%	42.1%
ADAM	750	69.0%	73.3%	51.3%	56.0%	79.2%	73.7%	100.0%	71.7%	100.0%	57.1%	100.0%	42.1%
ADAM	1000	69.0%	73.3%	51.3%	56.0%	79.2%	73.7%	100.0%	71.7%	100.0%	57.1%	100.0%	42.1%
LBFGS	200	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
LBFGS	500	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
LBFGS	750	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
LBFGS	1000	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
SGD	200	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	88.3%	80.0%	81.8%	81.8%	81.8%	47.4%
SGD	500	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	96.2%	73.3%	95.9%	60.0%	92.2%	47.4%
SGD	750	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	96.2%	73.3%	95.9%	60.0%	92.2%	47.4%
SGD	1000	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	98.7%	73.3%	98.7%	60.0%	97.4%	47.4%

Batch_size = 32

Solver	Iter	Tr_Acc(Raw)	Ts_Acc(Raw)	Tr_Prc(Raw)	Ts_Prc(Raw)	Tr_Rec(Raw)	Ts_Rec(Raw)	Tr_Acc(Scale)	Ts_Acc(Scale)	Tr_Prc(Scale)	Ts_Prc(Scale)	Tr_Rec(Scale)	Ts_Rec(Scale)
ADAM	200	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	97.9%	68.3%	97.4%	50.0%	96.1%	36.8%
ADAM	500	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	100.0%	70.0%	100.0%	53.8%	100.0%	36.8%
ADAM	750	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	100.0%	70.0%	100.0%	53.8%	100.0%	36.8%
ADAM	1000	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	100.0%	70.0%	100.0%	53.8%	100.0%	36.8%
LBFGS	200	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
LBFGS	500	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
LBFGS	750	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
LBFGS	1000	32.2%	31.7%	32.2%	31.7%	100.0%	100.0%	100.0%	71.7%	100.0%	58.3%	100.0%	36.8%
SGD	200	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	87.4%	78.3%	81.3%	80.0%	79.2%	42.1%
SGD	500	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	93.7%	75.0%	90.8%	64.3%	89.6%	47.4%
SGD	750	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	95.4%	73.3%	93.4%	60.0%	92.2%	47.4%
SGD	1000	67.8%	68.3%	0.0%	0.0%	0.0%	0.0%	97.5%	73.3%	97.3%	60.0%	94.8%	47.4%

Berdasarkan hasil dari berbagai skenario, didapatkan hasil bahwa model sgd dengan *batch size* 20 dengan scaling adalah yang terbaik. Hal ini dapat dilihat dari hasil *accuracy* dan *precision* yang *good fit*, dimana pada dunia medis *accuracy* dan *precision* adalah yang paling penting dan banyak digunakan sebagai evaluasi karena ingin mengetahui performa model dan juga baik dalam memprediksi pasien yang memiliki risiko besar meninggal karena gagal jantung.

Activation Function yang digunakan pada *hidden layer* adalah ReLU karena tidak ada fitur yang dapat memiliki nilai negatif, sehingga ketika terdapat nilai negatif maka akan langsung diubah jadi 0, jika tidak maka nilainya tetap akan diteruskan.

IV. REFERENSI

- [1] T. A. McDonagh, et al., "2021 ESC Guidelines for the diagnosis and treatment of acute and chronic heart failure," *European Heart Journal*, 2021.
- [2] C. W. Tsao, et al., "Heart Disease and Stroke Statistics—2023 Update," *Circulation*, 2023.
- [3] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., 2009.
- [4] M. Shanker, et al., "Effect of data standardization on neural network training," *Omega*, 1996.
- [5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv*, 2014.
- [6] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., 2006.